

Deepseek Chat

Please give me a Python program which does the following: I have a dump of German newspaper websites. The user selects a folder and the Python program crawls all ".html" files in that folder and all subfolders. It extracts the news articles from those html files and crawls them for a match with 2 REGEXes , in the code "regex1" and "regex2" (the code shall be written so that regex patterns can be added later on...). It writes the sentences in those articles which contain a match in an XML output file specified by the user into an "entry" tag with the filename of the actual file processed as an attribute. Please make sure that the output file is written "on the fly" so memory consumption stays minimal!



German Newspaper Article Regex Matcher

Here's a Python program that processes HTML files from German newspapers, applies regex patterns, and writes matches to an XML file efficiently:

python

Copy Download

```
import os
import re
import argparse
from bs4 import BeautifulSoup
import xml.etree.ElementTree as ET
from xml.dom import minidom

def process_html_file(file_path, regexes):
    """Process a single HTML file and return matches for all regexes"""
    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            soup = BeautifulSoup(f, 'htmlparser')

            # Remove script and style elements
            for script in soup(["script", "style"]):
                script.decompose()

            # Get all text and split into sentences (very simple approach)
            text = soup.get_text(separator=' ', strip=True)
            sentences = re.split(r'(?<!\w\.\w.)(?<![A-Z][a-z]\.)(?<=\.|?|!)\s', text)

            matches = []
            for sentence in sentences:
                for regex in regexes:
                    match = re.search(regex, sentence)
                    if match:
                        matches.append((sentence, match.group(1)))
```

```

        for sentence in sentences:
            for name, pattern in regexes.items():
                if pattern.search(sentence):
                    matches.append({
                        'sentence': sentence.strip(),
                        'pattern': name
                    })
            return matches
    except Exception as e:
        print(f"Error processing {file_path}: {str(e)}")
        return []

def write_xml_entry(xml_file, file_path, matches):
    """Write matches for a single file to the XML file"""
    if not matches:
        return

    # Create entry element
    entry = ET.Element('entry', {'file': os.path.basename(file_path), 'path': file_path})

    for match in matches:
        match_elem = ET.SubElement(entry, 'match', {'pattern': match['pattern']})
        match_elem.text = match['sentence']

    # Write to file
    xml_str = ET.tostring(entry, encoding='unicode')
    xml_file.write(xml_str + '\n')

def process_directory(root_dir, output_file, regexes):
    """Process all HTML files in directory and subdirectories"""
    with open(output_file, 'w', encoding='utf-8') as f:
        # Write XML header
        f.write('<?xml version="1.0" encoding="UTF-8"?>\n')
        f.write('<matches>\n')

        # Process each HTML file
        for dirpath, _, filenames in os.walk(root_dir):
            for filename in filenames:
                if filename.lower().endswith('.html'):
                    full_path = os.path.join(dirpath, filename)
                    matches = process_html_file(full_path, regexes)
                    write_xml_entry(f, full_path, matches)

        # Close root element
        f.write('</matches>\n')

def main():
    # Define your regex patterns here
    regex_patterns = {
        'regex1': re.compile(r'your_first_pattern_here', re.IGNORECASE),
        'regex2': re.compile(r'your_second_pattern_here', re.IGNORECASE)
        # Add more patterns as needed
    }

```

```

# Set up argument parser
parser = argparse.ArgumentParser(
    description='Process German newspaper HTML files and extract sentences matching given patterns.')
parser.add_argument('input_dir', help='Directory containing HTML files')
parser.add_argument('output_file', help='XML output file path')

args = parser.parse_args()

print(f"Processing directory: {args.input_dir}")
print(f"Output will be written to: {args.output_file}")

process_directory(args.input_dir, args.output_file, regex_patterns)
print("Processing complete.")

if __name__ == '__main__':
    main()

```

How to Use This Program:

1. Save the code to a file (e.g., `newspaper_matcher.py`)
2. Replace `'your_first_pattern_here'` and `'your_second_pattern_here'` with your actual regex patterns
3. Run the program from command line:

text

 Copy  Download

```
python newspaper_matcher.py /path/to/html/folder /path/to/output.xml
```

Key Features:

1. Memory Efficiency:

- Processes files one at a time
- Writes to XML file incrementally ("on the fly")
- Doesn't load all files into memory at once

2. Flexible Regex Handling:

- Easy to add more patterns by extending the `regex_patterns` dictionary
- Each match is tagged with which pattern it matched

3. Robust HTML Processing:

- Uses BeautifulSoup to parse HTML and extract text
- Removes scripts and styles to focus on content
- Basic sentence splitting (you may need to adjust for German text)

4. Structured Output

- XML format with clear structure
- Each entry includes the source filename
- Each match is tagged with the pattern that matched

Note: For processing German text, you might want to enhance the sentence splitting logic to better handle German language specifics. The current version uses a simple approach that works for many cases but may need adjustment for your specific content.

Good! Now give me a Python regex which matches a word token when an "und" and the same word token with "Innen" or "innen" added follows to it!



Here's a Python regex pattern that matches word tokens where the base word is followed by "und" and the same word with "Innen" or "innen" added:

```
python 📄 Copy 📄 Download  
  
import re  
  
pattern = re.compile(r'\b(\w+)\b\s+und\s+\b\1(?:innen|Innen)\b', re.IGNORECASE)  
  
# Example usage:  
text = "Bürger und Bürgerinnen, Lehrer und Lehrerinnen, Schüler und SchülerInnen"  
matches = pattern.findall(text)  
print(matches) # Output: ['Bürger', 'Lehrer', 'Schüler']
```

Explanation of the regex:

1. `\b(\w+)\b` - Matches a whole word and captures it in group 1
 - `\b` - Word boundary
 - `(\w+)` - Captures one or more word characters (letters, digits, underscores)
2. `\s+und\s+` - Matches the word "und" with whitespace on both sides
3. `\b\1(?:innen|Innen)\b` - Matches the same word from group 1 followed by either "innen" or "Innen"
 - `\1` - Backreference to the first captured group (the base word)
 - `(?:innen|Innen)` - Non-capturing group for either lowercase or capitalized suffix

- `\b` - Word boundary
- 4. `re.IGNORECASE` - Makes the matching case insensitive (optional, depending on your needs)

Examples it will match:

- "Bürger und Bürgerinnen"
- "Lehrer und LehrerInnen"
- "Student und Studentinnen"
- "Ärztin und Ärztinnen"

Notes:

1. The pattern handles both lowercase "innen" and capitalized "Innen"
2. It matches the same base word before and after "und" (using backreference `\1`)
3. If you need to handle more complex word forms or compound words, the pattern might need adjustment
4. For German text specifically, you might want to consider adding support for Umlauts and other special characters by modifying `\w` to `[a-zA-ZäöüÄÖÜß]` if needed

Good! Now give me a Python REGEX which matches a word token, when a single "*" is within that string!



Here's a Python regex that matches a word token containing exactly one asterisk (`*`) within it:

```
python 📄 Copy ⬇️ Download  
  
import re  
  
pattern = re.compile(r'\b\w*%\w*\b')  
  
# Example usage:  
text = "This is a test*string with *single* asterisks and multiple**stars and some*more*examples"  
matches = pattern.findall(text)  
print(matches) # Output: ['test *string', '*single *', 'some *more *examples']
```

Explanation of the regex:

1. `\b` - Word boundary (start of a word)

2. `\w*` - Zero or more word characters (letters, digits, underscores)
3. `\*` - A literal asterisk character (needs to be escaped with `\`)
4. `\w*` - Zero or more word characters
5. `\b` - Word boundary (end of a word)

If you want to match words with exactly one asterisk (not more):

python

 Copy  Download

```
pattern = re.compile(r'\b[^\s]*\*[^\s]*\b')
```

This stricter version:

- `[^\s]*` - Matches any characters except asterisks or whitespace
- Ensures there's exactly one asterisk in the word

Example with the stricter version:

python

 Copy  Download

```
text = "test*string *single* multiple**stars some*more*examples word*"
matches = pattern.findall(text)
print(matches) # Output: ['test *string', '*single *', 'word *']
```

If you need to handle German characters (Umlauts, ß):

python

 Copy  Download

```
pattern = re.compile(r'\b[a-zA-ZäöüÄÖÜß]*[a-zA-ZäöüÄÖÜß]*\b', re.IGNORECASE)
```

This version explicitly includes German special characters in the character class.